benchmax — controlled benchmark of LLM serving backends

Compares **vLLM**, **llama.cpp**, and other OpenAI-compatible serving backends under identical, deterministic workloads, measuring time-to-first-token, inter-token latency, throughput, memory, and output quality under quantization.

The goal is **not** to crown a winner. These backends optimize different objectives under different constraints, and on a single 8 GiB consumer GPU the constraints dominate the architecture. The deliverable is a set of reproducible measurements plus an explicit account of what each number does and does not license you to conclude.

Quick start

Everything runs inside WSL2 (vLLM has no native Windows support, and running both backends in the same environment is what makes the comparison fair).

```
# one-time setup (see scripts/ for the individual stages)
bash scripts/stage1_bootstrap.sh        # venv + cmake/ninja
bash scripts/stage2b3_cuda_from_redist.sh # rootless CUDA toolkit for the build
bash scripts/stage2a_install_vllm.sh    # vLLM
bash scripts/stage2c_build_llamacpp.sh  # llama.cpp with CUDA
bash scripts/stage3_bench_venv.sh       # harness venv

# then
python -m benchmax.cli list          # planned matrix
python -m benchmax.cli preflight     # which model files exist
python -m benchmax.cli download      # fetch artifacts
python -m benchmax.cli run           # execute
python -m benchmax.cli report        # tables + figures
```

`python -m benchmax.cli selftest` validates the measurement core against a mock endpoint with known injected latencies — run it first if you change anything in `benchmax/core.py`.

Why these choices

The `/v1/completions` endpoint, not chat. Chat endpoints apply each model's chat template. Qwen3-style hybrid-thinking models inject a reasoning block whose length varies per model and per prompt, which would silently change token counts across the matrix. Completions keeps the workload byte-identical everywhere.

Backends as separate server processes. An in-process Python API would hide exactly the differences under test — scheduling, continuous batching, KV cache management, kernel dispatch.

Client-side timing on a monotonic clock, at chunk arrival. This is the external view of latency, including HTTP and event-loop overhead. At sub-100 ms deltas that overhead is not negligible, so `selftest` measures it rather than assuming it away.

Aggregate throughput against group wall time, not the sum of per-request times. Summing would credit a concurrent run with more throughput than the server actually delivered.

Warmup pass discarded per cell. The first CUDA-graph capture, autotuning sweep and allocator warmup a server performs would otherwise land entirely on the first measured request.

Quantization is not a format identity. llama.cpp consumes GGUF (`Q4_K_M` / `Q8_0` / `F16`); vLLM's fast GPU kernels target AWQ and BF16. "4-bit" means the backend's preferred 4-bit scheme. These are different algorithms with different error characteristics, so part of any quality delta between them belongs to the quantizer rather than the backend. Every cell records its exact scheme string.

Unquantized is the reference. Perplexity and multiple-choice accuracy are reported relative to each model's own 16-bit run, not to some external baseline.

Unavailable cells are recorded, not dropped. No curated INT8 checkpoint exists for several models and no AWQ checkpoint exists for Qwen3-0.6B/1.7B, so those cells are written as `skipped` with a reason. That gap is itself a finding about the quantization surface each project supports.

Backend variants, not just backends. `llamacpp`, `vllm`, `vllm_gguf` and `ollama` are separate adapters. `vllm_gguf` was designed to hold weights fixed while varying only the backend; it is present but **skipped**, because vLLM 0.31 removed GGUF support. That negative result is more informative than the comparison would have been.

A lockfile prevents concurrent runs. Two benchmark servers driving one GPU silently contaminate each other's latency numbers, and nothing in the output makes that visible. The runner refuses to start while another holds the lock.

Client connection pool is sized above the largest workload. `requests` defaults to 10 connections; at 32 concurrent requests that put client-side queuing inside the TTFT measurement, reporting idle-server latency as seconds long. Verified against a mock endpoint with known injected latency.

What is measured

Metric	Definition
TTFT	arrival of first token minus request send. Includes queueing, prefill and first decode step.
ITL	interval between consecutive token arrivals, reported p50/p95/p99.
TPOT	mean interval between tokens — the latency a streaming user feels per token.
Output throughput	total completion tokens ÷ group wall time.
Prefill throughput	total prompt tokens ÷ group wall time.
Goodput	tokens/s from requests meeting **both** TTFT ≤ 1000 ms and TPOT ≤ 100 ms. A backend can post excellent mean throughput while failing this.
Peak VRAM / host RSS	sampled at 2 Hz throughout each run; peaks reported.
Perplexity	from the server's own prompt logprobs over a fixed wikitext-2 slice, so it scores the weights actually resident in the server.
MC accuracy	60 fixed items, scored by top-logprob at one forward pass, with a Wilson interval.

Workloads

ID	Shape	Isolates
`w1_single_short`	1 req, short prompt, 128 out	interactive latency floor
`w2_single_prefill`	1 req, ~2k-token prompt, 64 out	prefill / TTFT under load
`w3_concurrent_8`	8 concurrent	light continuous batching
`w4_concurrent_32`	32 concurrent	aggregate throughput ceiling

Layout

```
benchmax/
  core.py         measurement: TTFT, ITL, TPOT, throughput, goodput
  backends.py     server lifecycle + GPU/host memory sampling
  workloads.py    deterministic prompt generation
  quality.py      perplexity + multiple-choice probe
  config.py       experiment matrix and quantization-scheme mapping
  models.py       artifact resolution and download
  runner.py       orchestrator
  report.py       tables + figures from raw records
  cloud.py        hosted-endpoint adapter
  mock_cloud.py   local OpenAI-compatible mock (test fixture)
scripts/          setup and orchestration shell scripts
results/raw/      append-only runs.jsonl + environment + versions
reports/          generated tables, figures
```

`results/raw/runs.jsonl` is the source of truth. Every aggregate is re-derived from it by `report.py`. If a number in a report looks wrong, fix the aggregation and re-run `report` — no inference needs to be repeated.

PDFs

`reports/pdf/` holds print-ready, self-contained PDFs — figures embedded, so they render on any machine without the repository alongside:

File	Pages	Contents
`REPORT.pdf`	11	the full report with figures
`DATA_APPENDIX.pdf`	22	every table: performance by workload/backend/quantization, frontier, quantization cost, lifecycle, memory, quality, capability gaps, matrix status, environment
`README.pdf`	4	this methodology document

Regenerate with `bash scripts/stage6_pdf.sh` (runs `report` first if you have not already). `scripts/md_to_pdf.py` renders Markdown to PDF using `markdown` plus WeasyPrint — no pandoc, LaTeX or LibreOffice, none of which are installed here. The intermediate HTML is kept at `reports/pdf/html/`.

Known limitations

- **The comparison is not weight-controlled.** vLLM 0.31 has **no GGUF support** (no `gguf` method in `QUANTIZATION_METHODS`, no `quantization.gguf` module, no `GGUFConfig`). llama.cpp

consumes GGUF; vLLM consumes safetensors + AWQ/FP8/GPTQ. The quantization surfaces are disjoint, so no configuration serves identical weights on both — every backend delta also confounds the quantizer and the artifact. This is the single biggest limitation and it is a property of the current releases, not the harness.

- **Single 8 GiB consumer GPU, shared with a running desktop.** Absolute numbers are not representative of datacenter hardware; the desktop's own GPU use is a real noise source.
- **WSL2 caps guest RAM at 11 GiB**, which would bound any CPU-offload configuration.
- **n = 3 replicates.** Enough for the 4–60× effects reported; not enough to resolve differences under ~10%.
- **Perplexity is not comparable across the two backends** — llama.cpp's tool scores 2048-token windows, vLLM's echo path ~18k tokens. Within-backend, across-quantization comparisons are valid; cross-tool ones are not.
- **llama.cpp cannot serve prompt logprobs.** `llama-server` rejects `echo=true` and returns empty `logprobs`, so its perplexity was measured offline from the identical GGUF file and its multiple-choice probe was skipped.
- **The 16-bit reference is a ceiling reference, not a deployment option** for the 3B/4B models; those cells are excluded by the VRAM budget rather than allowed to OOM.
- **The MC probe is not informative** (accuracies at or below the 0.25 chance rate) and should not be used as a capability signal.
- **One contaminated cell was found and re-measured.** `llamacpp/Qwen3-1.7B` initially reported 8× slower per-token than the smaller 0.6B model, which is not physically possible; concurrent GPU load from an orphaned server was the cause. Re-measuring on a quiet machine changed TPOT from 35–76 ms to 6.7–15.5 ms. The 2 Hz GPU-utilisation sampler flagged it by reporting 91–93% utilisation at batch size 1. Superseded records are archived, not deleted. **Treat any physically implausible latency figure with suspicion.**
- **No cloud measurements.** No provider credentials were available. The cloud adapter is implemented and verified against a local mock; add a key and run `python -m benchmax.cli cloud run` to populate that arm.

See `reports/REPORT.md` for results, the full trade-off analysis, and the backend-selection guide.